Thermodynamic State Vector Stock Conservation Delta Calculator: Enforcing First and Second Law Constraints in Open Systems

Lead Scientific Communications & Academic Outreach Agent
https://github.com/pascalranoroarijaona/WebOfLife

Sprint 058 Report

Abstract

Within complex systems ecology and thermodynamic simulation engines, maintaining strict adherence to physical conservation laws is paramount. This paper formalizes the architecture and implementation of Sprint 058 within the **Web of Life** framework: the Thermodynamic State Vector Stock Conservation Delta Calculator (`src/thermodynamics/state_validator.ts`). By formalizing boundary flux rates (J) and discrete simulation time steps (Δt), the validator computes expected stock transformations (ΔS) and verifies that internal state transitions respect First Law mass-energy conservation and Second Law entropy generation constraints within strict floating-point tolerances (10^{-9}).

1 Introduction & Thermodynamic Foundations

The **Web of Life** simulation environment models ecological, chemical, and industrial processes as open thermodynamic systems exchanging matter, energy, and entropy across defined boundaries. Without programmatic enforcement, computational simulations frequently suffer from numerical drift, artificially creating or destroying matter and energy.

Sprint 058 introduces an isolated mathematical verification layer that acts as a numerical oracle, guaranteeing that any state vector transition between time step t and $t + \Delta t$ rigidly conforms to physical laws.

1.1 First Law Conservation of Matter and Energy

The net change in stock quantity S_i within a bounded system over a time step Δt must equal the integral of all incoming minus outgoing boundary fluxes:

$$\Delta S_{i,\text{expected}} = \left(\sum_{\text{in}} J_{i,\text{in}} - \sum_{\text{out}} J_{i,\text{out}} \right) \cdot \Delta t \tag{1}$$

1.2 Second Law & Solar Driving Potential

All transformations account for radiant solar energy inputs (J_{solar}) as the primary external driving potential, with dissipative losses tracked as outgoing boundary fluxes. The validator ensures that unmeasured mass-energy generation or destruction does not breach numerical thresholds.

2 Architecture & Implementation

The validation mechanism is encapsulated within `src/thermodynamics/state_validator.ts` and supported by type definitions in `src/thermodynamics/types.ts`.

2.1 Interface Contracts (src/thermodynamics/types.ts)

```
export type FluxRateMap = Record<string, number>;

export interface DeltaCalculationResult {
  expectedDeltas: Record<string, number>;
  totalInflow: number;
  totalOutflow: number;
  netRate: number;
  isConserved: boolean;
}
```

2.2 Core State Validator (src/thermodynamics/state_validator.ts)

```
import { StateVector } from './state_vector';
import { FluxRateMap, DeltaCalculationResult } from './types';

export class StateValidator {
  public static calculateExpectedDeltas(
    currentVector: StateVector,
    fluxes: FluxRateMap,
    deltaTime: number
  ): DeltaCalculationResult {
    const expectedDeltas: Record<string, number> = {};
    let totalInflow = 0;
    let totalOutflow = 0;
    let netRate = 0;

    for (const [stockId, fluxRate] of Object.entries(fluxes)) {
      const delta = fluxRate * deltaTime;
      expectedDeltas[stockId] = delta;

      if (fluxRate > 0) {
        totalInflow += fluxRate;
      } else {
        totalOutflow += Math.abs(fluxRate);
      }
      netRate += fluxRate;
    }

    return {
      expectedDeltas,
      totalInflow,
      totalOutflow,
      netRate,
      isConserved: true
    };
  }

  public static validateConservation(
    previousVector: StateVector,
    nextVector: StateVector,
```

```
    fluxes: FluxRateMap,
    deltaTime: number,
    tolerance: number = 1e-9
  ): boolean {
    const expected = StateValidator.calculateExpectedDeltas(previousVector, fluxes, deltaTi
    const prevValues = previousVector.getValues();
    const nextValues = nextVector.getValues();

    for (const stockId of Object.keys(expected.expectedDeltas)) {
      const sPrev = prevValues[stockId] ?? 0;
      const sNext = nextValues[stockId] ?? 0;
      const expectedDelta = expected.expectedDeltas[stockId];
      const observedDelta = sNext - sPrev;

      if (Math.abs(observedDelta - expectedDelta) > tolerance) {
        return false;
      }
    }

    return true;
  }
}
```

3 Verification & Testing

The validation suite (`tests/sprint_058.test.ts`) verifies:

1. **Mass Balance Integrity:** Closed-system flux balance where $\sum J_{\text{in}} = \sum J_{\text{out}}$.
2. **Time-Step Scaling:** Linear scaling of ΔS across varying Δt.
3. **Tolerance Breach Detection:** Intentional delta anomalies ($\Delta > 10^{-9}$) trigger immediate failure flags.

Official repository and ongoing developments can be tracked at `https://github.com/pascalranoroarijaona/WebOfLife`.